unext | manipal Academy of BFSI

Your AGENTS.md — the Contract That Builds Your Use Case

How to write the AGENTS.md that steers Codex to build safe, scaffolded agent systems

Client: Participant Guide · Philips MR

Trainer: Advanced Agentic AI Bootcamp

Date: 2-day hands-on

PART 1

What AGENTS.md Is — and Why It Decides Your Two Days

Codex reads AGENTS.md automatically every time it works in your project. It is the single most important file you write — before you write any code.

WHAT IT IS

AGENTS.md in One Minute

What it is

- **A plain-Markdown file at your project root**
- Codex (running GPT/Claude Models) reads it automatically, every turn
- It is the shared standard — the same file other agentic coding tools read
- It is the context + rules that steer how the coding agent builds
- Think of it as the spec that drives spec-driven development

Why it decides your two days

- **A good AGENTS.md → the agent builds scaffolded, guardrailed code by construction**
- A missing/weak one → you fight the agent and fix unsafe code all day
- It encodes the 10-point scaffold, the 4 guardrail layers, and the MCP rules once
- You write it first; then every prompt to Codex inherits it
- This is 'orchestration > syntax' in practice

The content and the program approach is strictly confidential. It is strictly forbidden to share any part of this program design approach with any third party

MENTAL MODEL

The Mental Model

AGENTS.md is to Codex what a good onboarding doc is to a new engineer. You wouldn't tell a new hire "just build it" with no architecture, conventions, or guardrails. Don't tell your coding agent that either.

You write AGENTS.md

architecture, scaffold rules, guardrails, MCP tools

→

Codex reads it

every turn, automatically — it's the standing context

→

You prompt in plain English

"build the Ingest agent" — it inherits all the rules

→

It builds to the contract

scaffolded + guardrailed by construction, not by luck

Write the contract once. Every build step inherits it. That is why we write AGENTS.md before any code.

The content and the program approach is strictly confidential. It is strictly forbidden to share any part of this program design approach with any third party

PART 2

The Structure — 13 Sections

A complete AGENTS.md for this programme has thirteen sections. Each one tells the coding agent something it must not guess.

STRUCTURE

The 13 Sections at a Glance

1 · Project	The one-line goal + definition of done
2 · Tech stack & runtime	Python, uv, FastAPI, the LiteLLM gateway, the coding agent, secrets rule
3 · Golden rules	Build-first, never blind-accept, the 6-step cycle, no secrets
4 · Architecture	Orchestrator + agents + typed hand-offs + bound the loop
5 · The 10-point scaffold	The non-negotiable harness code must pass by construction
6 · Guardrails (4 layers)	Where each layer sits + what it checks for THIS use case
7 · MCP server requirements	The tools to expose + auth + rate limit + audit + validation
8 · Project layout	The hexagonal structure that keeps the core pure
9 · Commands	build / run / test / guardrail self-test
10 · Definition of Done	Deliverable + red-team check + README current + trace check green
11 · Do / Don't	Explicit guardrails for the coding agent
12 · Documentation (README.md)	Create a README and keep it in sync with the code, every step
13 · SDD & Traceability	Spec-first: every behaviour gets a SPEC ID, carried spec → code → test → PR

The content and the program approach is strictly confidential. It is strictly forbidden to share any part of this program design approach with any third party

SECTION 5 · THE HEART

The 10-Point Scaffold — Non-Negotiable

This is the harness. Tell the agent to make all ten true by construction — code that fails a check is not done:

- **1 · Type hints on every boundary**
- 2 · LLM output → Pydantic v2 strict model
- 3 · Async I/O — never block the event loop
- 4 · Typed resilient client (retry + jitter)
- 5 · Provider behind a port, not in the core
- **6 · Tool-call args validated BEFORE execution**
- 7 · Destructive tools: business-rule + auth guards
- 8 · FastAPI request AND response models
- 9 · Contract tests as a CI gate (mock the LLM)
- 10 · Pure core — no SDK / SQL / framework

The one-line gate: if all ten tick, the harness is up — the agent can build fast without a collapse. Treat the model as a flaky vendor; the harness is the stable thing you ship.

The content and the program approach is strictly confidential. It is strictly forbidden to share any part of this program design approach with any third party

The Four Guardrail Layers

Name all four in AGENTS.md, and say what each checks for YOUR use case — the agent wires them at every seam:

L1 · Input (pre-hook)	*before the agent reasons*	validate + classify input; treat tool/doc content as untrusted
L2 · Tool-call interceptor	*wraps every MCP tool call*	schema check + call budget + tool allowlist
L3 · Output (post-hook)	*before output acts / returns*	PII / secret / brand / factual scan
L4 · Governance gate	*before a consequential action*	human approval + full audit log

Defence in depth: no single layer is trusted alone. All four compose around every agent.

The content and the program approach is strictly confidential. It is strictly forbidden to share any part of this program design approach with any third party

MCP Server Build Requirements

What to specify

- **HTTP transport (FastAPI) — build for deploy from day one**
- List every tool the agents may call
- Mark which tools are read-only vs destructive
- Every tool: typed strict request/response
- Auth on every call · rate limit per caller · audit log
- Validated tool calls (check 6) + guarded destructive tools (check 7)

Example — a tool entry

AGENTS.md — §7 tools

```
Tools to expose (CS1):
- file_reader       (read-only)
- template_library  (read-only)
- version_control   (read-only;
    no writes without L4)
- approval_workflow
    DESTRUCTIVE -> auth +
    human confirmation (check 7)

Every tool: typed I/O, auth,
rate limit, audit log.
```

The content and the program approach is strictly confidential. It is strictly forbidden to share any part of this program design approach with any third party

README.md — Document What You Build

The rule

- **Create a README.md at the project root — and keep it updated**
- Codex must update the matching README section whenever it creates or changes a component
- If you change the code, update the README in the SAME step
- A stale README is a defect — it's in the Definition of Done
- Start from the provided README_SKELETON.md

What the README must contain

- 1 · What this is — one-paragraph overview
- 2 · Architecture — agents, orchestrator, MCP server, flow
- 3 · Prerequisites · 4 · Setup (uv sync, .env)
- 5 · How to run — copy-paste commands
- 6 · MCP tools — table: tool · does · read/destructive · gate
- 7 · Guardrails — the 4 layers and what each checks
- 8 · Env vars · 9 · Testing
- 10 · Build log — dated list of what's built, updated each session

The content and the program approach is strictly confidential. It is strictly forbidden to share any part of this program design approach with any third party

Spec-Driven Development & Traceability

Build spec-first. No code without a spec ID — and that one ID is carried all the way to the PR, so spec → code → test → PR is greppable by a single string:

The method — every behaviour

- **1 · Write the spec first**
- specs/SPEC-<CS>-NNN.md — small + pass/fail testable
- **2 · Implement + tag the code**
- # [SPEC-<CS>-NNN] on the relevant line(s)
- **3 · Write the test with the ID**
- def test_spec_<cs>_<nnn>_<behaviour>()
- **4 · Carry the ID into the PR title**
- feat(...): ... [SPEC-<CS>-NNN]

One ID · four anchors

SPEC-CS3-007

```
Spec   specs/SPEC-CS3-007.md
         "no prod change without L4 gate"
Code   # [SPEC-CS3-007] L4 gate blocks
         apply without human approval
Test   def test_spec_cs3_007_apply_
         blocked_without_approval()
PR     feat(gov): human gate [SPEC-CS3-007]
```

Traceability check in CI: scripts/trace_check.py — every spec ID must appear in ≥1 code file AND ≥1 test. An orphan spec, or an ID with no spec, FAILS the build. grep one ID → spec + code + test + PR.

The content and the program approach is strictly confidential. It is strictly forbidden to share any part of this program design approach with any third party

PART 3

The Skeleton — Copy This to Start

Blank AGENTS.md + README.md + SPEC skeletons you fill in per use case. We provide these plus three filled AGENTS.md versions (CS1, CS2, CS3) as a base.

SKELETON

AGENTS.md Skeleton (1 of 2)

AGENTS.md

```
# AGENTS.md — <USE CASE NAME>

## 1. Project
- Goal (one line): <what this system does>
- Use case: <CS1/CS2/CS3>   Owner: <name>

## 2. Tech stack & runtime
- Python 3.11 · uv (uv sync --frozen) · FastAPI (async) · <LangGraph/CrewAI>
- Runtime model: LiteLLM gateway http://<gateway>:4000/v1  (NO external APIs)
- Coding agent: Codex + GPT · Secrets from .env only

## 3. Golden rules for the coding agent
- Spec-first (§13) · build first · never blind-accept · 6-step cycle · scaffold non-negotiable

## 4. Architecture to build
- Orchestrator -> domain agents <list> with typed hand-offs; bound the loop

## 5. The 10-point scaffold — MUST pass  (list all ten)
## 6. Guardrails — the 4 layers  (L1 input · L2 tool · L3 output · L4 gate)
```

The content and the program approach is strictly confidential. It is strictly forbidden to share any part of this program design approach with any third party

SKELETON

AGENTS.md Skeleton (2 of 2)

AGENTS.md (continued)

```
## 7. MCP server build requirements
- HTTP transport; tools: <list, mark destructive>
- Every tool: typed I/O · auth · rate limit · audit · validated calls

## 8. Project layout  (hexagonal — keep the core pure)
  app/core/      PURE: models.py ports.py agents.py guardrails.py
  app/adapters/  llm_client.py  mcp_tools.py
  app/api/       server.py  (FastAPI MCP server)
  app/tests/     contract + property tests (LLM mocked)
  .github/workflows/security.yml   (CI gate)

## 9. Commands
  uv sync --frozen
  uv run uvicorn app.api.server:app --reload
  uv run pytest

## 10. Definition of Done  (pipeline · 4 layers ·   attacks blocked · CI green · README + trace)
## 11. Do / Don't  (explicit guardrails for the coding agent)
## 12. Documentation — README.md  (create + keep in sync with the code, every step)
## 13. SDD & Traceability  (spec-first · SPEC-<CS>-NNN carried spec -> code -> test -> PR)
```

The content and the program approach is strictly confidential. It is strictly forbidden to share any part of this program design approach with any third party

PART 4

Using It Across the Two Days

One AGENTS.md per use-case project. Three are provided as a base — start from the matching one, fill the <placeholders>, then build.

WORKFLOW

The Build Loop With Codex

Step by step

- **1 · Put AGENTS.md at your project root**
- Codex reads it automatically — no flag needed
- **2 · Fill the placeholders**
- <gateway> IP, <model>, your name, framework choice
- **3 · Prompt in plain English, one agent at a time**
- "Build the Ingest agent and its pre-hook"
- **4 · Review every change before you accept it**
- the scaffold rules make the diff easy to trust
- **5 · Run the loop: Define → … → Attack → Commit**
- re-run the red-team checks; commit when green

Keep AGENTS.md alive

- **Edit it as the design changes — it's the source of truth**
- If the agent does something wrong, the fix usually belongs in AGENTS.md, not the prompt
- Add a project-specific Do/Don't the moment you hit a sharp edge
- A short, accurate AGENTS.md beats a long, stale one

The content and the program approach is strictly confidential. It is strictly forbidden to share any part of this program design approach with any third party

TIPS

A Good AGENTS.md vs a Weak One

✓ A good one

- **Specific: names the agents, tools, and what each guardrail checks**
- Lists the 10 scaffold points explicitly
- Marks which tools are destructive
- Has a concrete Definition of Done + attack
- Short, accurate, and kept up to date
- Tells the agent what NOT to do

✗ A weak one

- **Vague: "build a documentation system with guardrails"**
- No scaffold rules → unsafe code by default
- No tool list → the agent invents tools
- No destructive-tool marking → unguarded actions
- No DoD → you never know when it's done
- Stale → the agent follows the wrong contract

The content and the program approach is strictly confidential. It is strictly forbidden to share any part of this program design approach with any third party

Write the Contract First

A clear AGENTS.md is the difference between the agent building WITH you and against you. Start from your use-case file, fill the placeholders, and build.

The content and the program approach is strictly confidential. It is strictly forbidden to share any part of this program design approach with any third party